

SQLAlchemy 2.x Cheatsheet — Complete Reference

Setup & Session

```
from sqlalchemy import create_engine, select, insert, update, delete, text, func,
and_, or_, not_, exists, distinct
from sqlalchemy.orm import sessionmaker, DeclarativeBase, Session, relationship,
selectinload, joinedload, subqueryload, contains_eager

# Engine
engine = create_engine(
    "postgresql+psycopg2://user:pass@localhost/dbname",
    pool_pre_ping=True,          # auto-reconnect on stale connections
    pool_size=10,
    max_overflow=20,
    echo=False,                  # set True to log all SQL
)

# Session factory
SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)

# Base model class
class Base(DeclarativeBase):
    pass

# Create all tables
Base.metadata.create_all(bind=engine)

# Drop all tables
Base.metadata.drop_all(bind=engine)
```

FastAPI dependency

```
def get_db():
    db = SessionLocal()
    try:
        yield db
        db.commit()
    except Exception:
        db.rollback()
        raise
    finally:
        db.close()

# In route
@app.get("/users")
def list_users(db: Session = Depends(get_db)):
    ...
```

Async setup

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession,
async_sessionmaker
```

```

engine = create_async_engine("postgresql+asyncpg://user:pass@localhost/db",
echo=False)
AsyncSessionLocal = async_sessionmaker(engine, expire_on_commit=False)

async def get_db():
    async with AsyncSessionLocal() as db:
        try:
            yield db
            await db.commit()
        except Exception:
            await db.rollback()
            raise

```

Model Definition

```

from sqlalchemy import Column, Integer, String, Boolean, DateTime, ForeignKey,
Table, Enum, JSON, Text, Numeric
from sqlalchemy.sql import func
import enum

class RoleEnum(enum.Enum):
    user = "user"
    admin = "admin"
    mod = "mod"

# Association table for many-to-many
post_tags = Table(
    "post_tags", Base.metadata,
    Column("post_id", ForeignKey("posts.id"), primary_key=True),
    Column("tag_id", ForeignKey("tags.id"), primary_key=True),
)

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    email = Column(String(255), unique=True, nullable=False, index=True)
    role = Column(Enum(RoleEnum), default=RoleEnum.user)
    active = Column(Boolean, default=True)
    age = Column(Integer, nullable=True)
    bio = Column(Text, nullable=True)
    meta = Column(JSON, nullable=True)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), onupdate=func.now())

    # Relationships
    posts = relationship("Post", back_populates="author", cascade="all, delete-orphan")
    profile = relationship("Profile", back_populates="user", uselist=False,
cascade="all, delete-orphan")

class Post(Base):
    __tablename__ = "posts"

```

```

    id      = Column(Integer, primary_key=True, index=True)
    title   = Column(String(255), nullable=False)
    body    = Column(Text)
    published = Column(Boolean, default=False)
    author_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"))

    author = relationship("User", back_populates="posts")
    tags   = relationship("Tag", secondary=post_tags, back_populates="posts")

class Tag(Base):
    __tablename__ = "tags"
    id      = Column(Integer, primary_key=True)
    name    = Column(String(50), unique=True)
    posts   = relationship("Post", secondary=post_tags, back_populates="tags")

class Profile(Base):
    __tablename__ = "profiles"
    id      = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey("users.id"), unique=True)
    bio     = Column(Text)
    user    = relationship("User", back_populates="profile")

```

CRUD

Create

```

# Single insert
user = User(name="Alice", email="alice@example.com", role=RoleEnum.user)
db.add(user)
db.commit()
db.refresh(user)  # loads db-generated fields (id, created_at, etc.)

# Add multiple at once
db.add_all([User(name="Bob", email="b@x.com"), User(name="Carl", email="c@x.com")])
db.commit()

```

Read

```

# By primary key
user = db.get(User, user_id)  # returns None if not found

# Single row
stmt = select(User).where(User.email == "alice@example.com")
user = db.execute(stmt).scalar_one_or_none()  # None if missing, raises if 2+
user = db.execute(stmt).scalar_one()          # raises if 0 or 2+

# All rows
stmt = select(User)
users = db.execute(stmt).scalars().all()

# First row
user = db.execute(stmt).scalars().first()

```

Update

```
# Fetch-then-set
user = db.get(User, user_id)
user.name = "Alice Updated"
db.commit()
db.refresh(user)

# Core UPDATE (no fetch, one query)
stmt = update(User).where(User.id == user_id).values(name="Alice Updated")
db.execute(stmt)
db.commit()
```

Delete

```
# Fetch-then-delete
user = db.get(User, user_id)
db.delete(user)
db.commit()

# Core DELETE (no fetch, one query)
stmt = delete(User).where(User.id == user_id)
db.execute(stmt)
db.commit()
```

Query & Filter

Fetch variants

```
db.execute(select(User)).scalars().all()           # list, [] if none
db.execute(select(User)).scalars().first()         # first or None
db.execute(select(User)).scalar_one()              # exactly 1, raises otherwise
db.execute(select(User)).scalar_one_or_none()      # None if missing, raises if 2+
db.execute(select(User)).scalars().unique().all()  # deduplicate (use after joins)

# Count
from sqlalchemy import func
db.execute(select(func.count()).select_from(User)).scalar()
```

Select specific columns

```
stmt = select(User.id, User.name, User.email)
rows = db.execute(stmt).all()
for row in rows:
    print(row.id, row.name)  # access by attribute name

# With label
stmt = select(User.name, func.count(Post.id).label("post_count"))\
    .join(Post, User.id == Post.author_id)\
    .group_by(User.id)
rows = db.execute(stmt).all()
```

Filter operators

```
# Equality & comparison
User.age == 30
User.age != 30
User.age > 18
User.age >= 18
User.age < 65
User.age.between(18, 65)

# String matching
User.name.like("%ali%")           # case-sensitive
User.name.ilike("%ali%")         # case-insensitive
User.email.startswith("a")
User.email.endswith("@gmail.com")
User.bio.contains("python")

# Null checks
User.deleted_at.is_(None)         # IS NULL
User.deleted_at.is_not(None)     # IS NOT NULL

# Membership
User.id.in_([1, 2, 3])
User.id.not_in([4, 5])

# Boolean composition
stmt = select(User).where(
    and_(
        User.active == True,
        or_(User.role == "admin", User.age > 30),
        not_(User.email.like("%spam%"))
    )
)

# Multiple .where() calls are ANDed
stmt = select(User).where(User.active == True).where(User.role == "admin")
```

Dynamic / conditional filters

```
def search_users(db, name=None, role=None, active=None):
    stmt = select(User)
    if name:
        stmt = stmt.where(User.name.ilike(f"%{name}%"))
    if role:
        stmt = stmt.where(User.role == role)
    if active is not None:
        stmt = stmt.where(User.active == active)
    return db.execute(stmt).scalars().all()
```

Sorting, limit, offset

```
select(User).order_by(User.name)
select(User).order_by(User.name.asc())
select(User).order_by(User.created_at.desc())
select(User).order_by(User.role, User.name.desc())  # multi-column

# Pagination
page, per_page = 2, 10
```

```
stmt = select(User).order_by(User.id).limit(per_page).offset((page - 1) * per_page)
```

Aggregates & group by

```
db.execute(select(func.count(User.id))).scalar()
db.execute(select(func.sum(Order.total))).scalar()
db.execute(select(func.avg(Order.total))).scalar()
db.execute(select(func.min(User.age))).scalar()
db.execute(select(func.max(User.age))).scalar()

# Group by with having
stmt = select(User.role, func.count(User.id).label("cnt"))\
    .group_by(User.role)\
    .having(func.count(User.id) > 5)
rows = db.execute(stmt).all()
```

Distinct

```
stmt = select(distinct(User.role))
roles = db.execute(stmt).scalars().all()

stmt = select(User).distinct()
users = db.execute(stmt).scalars().all()
```

Joins

```
# Inner join (default)
stmt = select(User).join(User.posts)
stmt = select(User).join(Post, User.id == Post.author_id)

# Left outer join
stmt = select(User).outerjoin(User.posts)

# Join + filter
stmt = select(User)\
    .join(Post, User.id == Post.author_id)\
    .where(Post.published == True)\
    .distinct()

# Multi-table join
stmt = select(User)\
    .join(Post, User.id == Post.author_id)\
    .join(post_tags, Post.id == post_tags.c.post_id)\
    .join(Tag, Tag.id == post_tags.c.tag_id)\
    .where(Tag.name == "python")

# Explicit join condition with alias
from sqlalchemy.orm import aliased
AuthorAlias = aliased(User)
stmt = select(Post, AuthorAlias).join(AuthorAlias, Post.author_id ==
AuthorAlias.id)
```

Subqueries & exists

```
# EXISTS
stmt = select(User).where(
```

```

    exists().where(Post.author_id == User.id)
)

# NOT EXISTS
stmt = select(User).where(
    ~exists().where(Post.author_id == User.id)
)

# Scalar subquery
post_count = select(func.count(Post.id))\
    .where(Post.author_id == User.id)\
    .scalar_subquery()
stmt = select(User, post_count.label("posts"))

# Derived table (subquery as FROM)
subq = select(func.avg(Order.total).label("avg")).subquery()
stmt = select(Order).where(Order.total > subq.c.avg)

# CTE (common table expression)
cte = select(User).where(User.active == True).cte("active_users")
stmt = select(Post).where(Post.author_id.in_(select(cte.c.id)))

```

has() and any() — filter via relationship

```

# Users with at least one published post
stmt = select(User).where(User.posts.any(Post.published == True))

# Posts whose author is admin
stmt = select(Post).where(Post.author.has(User.role == "admin"))

# Users with NO posts
stmt = select(User).where(~User.posts.any())

```

Bulk Operations

Bulk insert

```

# add_all – ORM objects, fires events/hooks
db.add_all([
    User(name="Alice", email="a@x.com"),
    User(name="Bob", email="b@x.com"),
])
db.commit()

# Core insert – fastest, no ORM overhead
db.execute(insert(User), [
    {"name": "Alice", "email": "a@x.com"},
    {"name": "Bob", "email": "b@x.com"},
])
db.commit()

# Return inserted rows (PostgreSQL)
result = db.execute(
    insert(User).returning(User.id, User.email),
    [{"name": "Alice", "email": "a@x.com"}]
)

```

```
)  
rows = result.all()
```

Upsert (insert or update on conflict)

```
# PostgreSQL  
from sqlalchemy.dialects.postgresql import insert as pg_insert  
  
stmt = pg_insert(User).values(email="alice@x.com", name="Alice")  
stmt = stmt.on_conflict_do_update(  
    index_elements=["email"],          # conflict column(s)  
    set_={"name": stmt.excluded.name}  # what to update  
)  
db.execute(stmt)  
db.commit()  
  
# SQLite  
from sqlalchemy.dialects.sqlite import insert as sqlite_insert  
stmt = sqlite_insert(User).values(email="alice@x.com", name="Alice")  
stmt = stmt.on_conflict_do_update(  
    index_elements=["email"],  
    set_={"name": stmt.excluded.name}  
)  
db.execute(stmt)  
db.commit()  
  
# Insert or ignore (do nothing on conflict)  
stmt = pg_insert(User).values(...).on_conflict_do_nothing()  
db.execute(stmt)  
db.commit()
```

Bulk update

```
# Core UPDATE – one query, no fetch  
stmt = update(User)\  
    .where(User.active == False)\  
    .values(role="guest", updated_at=func.now())  
db.execute(stmt)  
db.commit()  
  
# Update many rows with different values (executemany)  
db.execute(update(User), [  
    {"id": 1, "name": "Alice Updated"},  
    {"id": 2, "name": "Bob Updated"},  
)  
db.commit()  
  
# Correlated update (update from another table)  
stmt = update(User)\  
    .where(User.id == Post.author_id)\  
    .where(Post.published == False)\  
    .values(active=False)  
db.execute(stmt)  
db.commit()
```

Bulk delete

```
# Core DELETE – one query
stmt = delete(User).where(User.active == False)
db.execute(stmt)
db.commit()

# Delete with returning (PostgreSQL)
stmt = delete(User).where(User.active == False).returning(User.id)
deleted_ids = db.execute(stmt).scalars().all()
db.commit()
```

Stream large result sets (avoid OOM)

```
# yield_per – iterate in chunks
result = db.execute(select(User).execution_options(yield_per=1000))
for partition in result.partitions(1000):
    for row in partition:
        process(row)
```

Relationships

Model setup patterns

```
# One-to-many
class User(Base):
    posts = relationship("Post", back_populates="author", cascade="all, delete-orphan")

class Post(Base):
    author_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"))
    author = relationship("User", back_populates="posts")

# One-to-one
class User(Base):
    profile = relationship("Profile", back_populates="user", uselist=False, cascade="all, delete-orphan")

class Profile(Base):
    user_id = Column(Integer, ForeignKey("users.id"), unique=True)
    user = relationship("User", back_populates="profile")

# Many-to-many
post_tags = Table("post_tags", Base.metadata,
    Column("post_id", ForeignKey("posts.id"), primary_key=True),
    Column("tag_id", ForeignKey("tags.id"), primary_key=True),
)
class Post(Base):
    tags = relationship("Tag", secondary=post_tags, back_populates="posts")
class Tag(Base):
    posts = relationship("Post", secondary=post_tags, back_populates="tags")

# Self-referential (tree / org chart)
class Category(Base):
    id = Column(Integer, primary_key=True)
```

```

    parent_id = Column(Integer, ForeignKey("categories.id"), nullable=True)
    parent = relationship("Category", remote_side=["id"],
back_populates="children")
    children = relationship("Category", back_populates="parent")

```

Cascade options

```

# ORM cascades (set on relationship())
cascade="save-update, merge"           # default
cascade="all"                           # save-update, merge, delete, expunge
cascade="all, delete-orphan"           # also deletes child when removed from parent

# DB-level cascade (set on ForeignKey)
Column(Integer, ForeignKey("users.id", ondelete="CASCADE"))    # DB deletes
children
Column(Integer, ForeignKey("users.id", ondelete="SET NULL"))    # DB sets null
Column(Integer, ForeignKey("users.id", ondelete="RESTRICT"))    # DB blocks delete

```

Loading strategies

```

# lazy="select" (default) – fires SELECT when attribute is accessed
# lazy="raise" – raises DetachedInstanceError on access (detect N+1 in dev)
# lazy="joined" – always join-loads (same as joinedload option)
# lazy="subquery" – always subquery-loads
# lazy="selectin" – always selectin-loads (recommended default for 2.x)

class User(Base):
    posts = relationship("Post", lazy="selectin")    # set globally on model

# --- Per-query overrides (preferred in FastAPI) ---

# joinedload – LEFT JOIN, best for *-to-one
stmt = select(User).options(joinedload(User.profile)).where(User.id == 1)
user = db.execute(stmt).unique().scalar_one()

# selectinload – SELECT IN, best for one-to-many collections (recommended)
stmt = select(User).options(selectinload(User.posts))
users = db.execute(stmt).scalars().all()

# subqueryload – similar to selectinload but uses a subquery (older style)
stmt = select(User).options(subqueryload(User.posts))

# Load multiple relationships at once
stmt = select(User).options(
    selectinload(User.posts),
    joinedload(User.profile),
)
users = db.execute(stmt).unique().scalars().all()

# Nested eager load (posts + their tags)
stmt = select(User).options(
    selectinload(User.posts).selectinload(Post.tags)
)

# contains_eager – when you join manually, tell ORM to populate from that join
stmt = select(User)\
    .join(User.posts)\
    .where(Post.published == True)\

```

```
.options(contains_eager(User.posts))
users = db.execute(stmt).unique().scalars().all()
```

Write operations on relationships

```
# Add to collection
user = db.get(User, 1)
user.posts.append(Post(title="Hello"))
db.commit()

# Remove from collection (cascade="delete-orphan" deletes the child)
user.posts.remove(some_post)
db.commit()

# Assign one-to-one
user.profile = Profile(bio="Developer")
db.commit()

# Many-to-many add/remove
post.tags.append(tag)
post.tags.remove(tag)
db.commit()

# Replace entire collection
user.posts = [Post(title="A"), Post(title="B")]
db.commit()
```

Association proxy

```
from sqlalchemy.ext.associationproxy import association_proxy

class Post(Base):
    tags = relationship("Tag", secondary=post_tags)
    tag_names = association_proxy("tags", "name") # read tag.name directly

post = db.get(Post, 1)
print(post.tag_names) # ["python", "fastapi"]
post.tag_names.append("sql") # creates Tag if needed
db.commit()
```

Transactions

```
db.commit() # persist all pending changes
db.rollback() # discard all pending changes
db.close() # return connection to pool

# Flush – push to DB without committing (makes IDs available mid-transaction)
db.flush()

# Savepoint (nested transaction)
with db.begin_nested():
    db.add(User(name="Temp"))
    # rolls back only this block on error, outer transaction continues
db.commit()
```

```
# Explicit transaction context manager
with db.begin():
    db.add(User(name="Alice"))
    # auto-commits on exit, rolls back on exception

# Expire / refresh
db.expire(user)      # mark stale – reloads on next access
db.refresh(user)     # reload from DB immediately
db.expire_all()      # expire every tracked object
db.expunge(user)     # remove from session entirely (stops tracking)
db.expunge_all()     # remove all objects from session
```

Raw SQL

```
# Parameterized query – always use named params, never f-strings
result = db.execute(text("SELECT * FROM users WHERE id = :uid"), {"uid": user_id})
rows = result.fetchall()
row = result.fetchone()
row = result.mappings().first()    # dict-like access: row["name"]

# Scalar value
count = db.execute(text("SELECT COUNT(*) FROM users")).scalar()
name = db.execute(text("SELECT name FROM users WHERE id = :id"), {"id": 1}).scalar_one()

# Write / DDL
db.execute(text("UPDATE users SET active = :a WHERE id = :id"), {"a": True, "id": 1})
db.commit()

db.execute(text("CREATE INDEX CONCURRENTLY IF NOT EXISTS ix_users_email ON
users(email)"))
db.commit()

# Map raw results to ORM objects
from sqlalchemy.orm import Session
users = db.execute(
    text("SELECT * FROM users WHERE active = true")
).mappings().all()
```

Unique shortcode pattern (race-condition safe)

```
from sqlalchemy.exc import IntegrityError

def create_unique_shortcode(db: Session) -> str:
    for _ in range(5):
        short_code = generate_random_code()    # your code generator
        try:
            entry = UrlMap(shortcode=short_code, ...)
            db.add(entry)
            db.flush()    # catch IntegrityError before full commit
            db.commit()
            return short_code
```

```

        except IntegrityError:
            db.rollback()
            raise RuntimeError("Could not generate a unique short code after 5 attempts")

# Only works if `shortcode` column has a UNIQUE constraint in DB:
# shortcode = Column(String(10), unique=True, nullable=False)

```

Inspection & utils

```

from sqlalchemy import inspect

# Check if table exists
inspector = inspect(engine)
inspector.has_table("users")
inspector.get_table_names()
inspector.get_columns("users")
inspector.get_indexes("users")
inspector.get_foreign_keys("users")

# Check session state of an object
from sqlalchemy import inspect as sa_inspect
state = sa_inspect(user)
state.pending      # added but not flushed
state.persistent   # in session and in DB
state.detached     # no longer bound to a session
state.deleted      # marked for deletion

# Get the underlying SQL of a statement
print(stmt.compile(engine))

```

Common pitfalls

Problem	Cause	Fix
DetachedInstanceError	Accessing lazy relationship after session closes	Use <code>selectinload</code> / <code>joinedload</code> in the query
MissingGreenlet in async	Using sync ORM calls inside async context	Use <code>AsyncSession</code> and <code>await db.execute(...)</code>
N+1 queries	Lazy loading in a loop	Eager load with <code>selectinload</code> or <code>joinedload</code>
duplicate key race condition	Pre-checking uniqueness instead of letting DB enforce it	Add <code>UNIQUE</code> constraint, catch <code>IntegrityError</code>
Stale data after commit	<code>expire_on_commit=True</code> (default) marks all objects stale	Call <code>db.refresh(obj)</code> or set <code>expire_on_commit=False</code>
greenlet_spawn error	Lazy load triggered inside async code	Set <code>lazy="raise"</code> in dev to catch these early

Problem	Cause	Fix
<code>unique()</code> not called after join	Duplicate rows when <code>joinedload</code> multiplies rows	Always call <code>.unique()</code> before <code>.scalars().all()</code> after <code>joinedload</code>